

一、某 C 语言程序有两个源文件：main.c 和 test.c，它们的内容如下图所示。

```
/* main.c */
1  #include <stdio.h>
2
3  float ar[0x100000] = { 1.0 };
4  extern void test();
5
6  void main()
7  {
8      float f;
9      test();
10
11     f = ar[0x2000];
12     printf("%f\n", f);
13 }
```

```
/* test.c */
1  struct R {
2      short n1, n2;
3  } ar[1024][1024];
4
5  void test() {
6      int i, j;
7      for( i = 0; i < 1024; i++ )
8          for( j = 0; j < 1024; j++ )
9          {
10             ar[i][j].n1 = j;
11             ar[i][j].n2 = i | 0xc000;
12         }
13 }
```

假设在 IA-32+Linux 平台上用 GCC 编译驱动程序处理，main.c 和 test.c 的可重定位目标文件名分别是 main.o 和 test.o，生成的可执行文件名为 test。回答下列问题或完成下列任务（提示：IA-32 为小端方式，字长为 32 位，按字节编址）。

1. 简述可执行文件 test 的生成过程。（4 分）

2. 假设 test.c 中数组 ar 在虚拟地址空间中的首址为 0x804c040，则数组元素 ar[4][4]存放在哪些虚拟地址中？对应各存储单元中的内容分别是什么（用十六进制形式表示）？（4 分）

依据下列图中 main.o 的反汇编代码，画出在执行 0x31 处的 call 指令后（在被调用函数返回前）main 函数栈帧的状态，要求给出栈帧中存放的内容及其所在的地址，并给出 ESP 所指的位置（地址和位置都表示为相对于 EBP 的偏移量形式，如 ebp-0xc 或 ebp-12）。

# main.o 部分反汇编结果 #	# test.o 部分反汇编结果 #
00000000 <main>:	00000000 <test>:
a: 55 push %ebp	5: be 00 00 00 00 mov \$0x0,%esi
b: 89 e5 mov %esp,%ebp	a: eb 3d jmp 49 <test+0x49>
d: 51 push %ecx	c: bb 00 00 00 00 mov \$0x0,%ebx
e: 83 ec 14 sub \$0x14,%esp	11: eb 2b jmp 3e <test+0x3e>
11: e8 fc ff ff ff call 12 <main+0x12>	13: 89 da mov %ebx,%edx
16: d9 05 00 80 00 00 flds 0x8000	15: 89 f0 mov %esi,%eax
1c: d9 5d f4 fstps -0xc(%ebp)	17: c1 e0 0a shl \$0xa,%eax
1f: d9 45 f4 flds -0xc(%ebp)	1a: 01 d8 add %ebx,%eax
22: 83 ec 04 sub \$0x4,%esp	1c: 66 89 14 85 00 00 00 00 mov %dx,0x0(,%eax,4)
25: 8d 64 24 f8 lea -0x8(%esp),%esp	24: 89 f0 mov %esi,%eax
29: dd 1c 24 fstpl (%esp)	26: 66 0d 00 c0 or \$0xc000,%ax
2c: 68 00 00 00 00 push \$0x0	2a: 89 c2 mov %eax,%edx
31: e8 fc ff ff ff call 32 <main+0x32>	2c: 89 f0 mov %esi,%eax
36: 83 c4 10 add \$0x10,%esp	2e: c1 e0 0a shl \$0xa,%eax
39: b8 00 00 00 00 mov \$0x0,%eax	31: 01 d8 add %ebx,%eax
3e: 8b 4d fc mov -0x4(%ebp),%ecx	33: 66 89 14 85 02 00 00 00 mov %dx,0x2(,%eax,4)
.....	3b: 83 c3 01 add \$0x1,%ebx
	3e: 81 fb ff 03 00 00 cmp \$0x3ff,%ebx
	44: 7e cd jle 13 <test+0x13>
	46: 83 c6 01 add \$0x1,%esi
	49: 81 fe ff 03 00 00 cmp \$0x3ff,%esi
	4f: 7e bb jle
	

3. 从上述 test.o 反汇编结果中的哪些指令能看出 IA-32 采用的是小端方式？为什么？（2 分）

4. 对照 test.c 和 test.o 的反汇编结果说明 test 函数中语句“ar[i][j].n2 = i | 0xc000”对应 test.o 中哪几条机器指令？逐一解释每条指令的功能。（5 分）

5. 指令 `cmp` 的执行将会影响 `EFLAGS` 寄存器中哪些常用标志？当 `test` 函数中的 `i`、`j` 变量均等于 10 时，`test.o` 中 `0x3e` 处的 `cmp` 指令的执行结果将如何影响下条 `jle` 指令？（6 分）
6. `test.o` 的反汇编代码中，`0x4f` 处的 `jle` 指令的第二字节为什么是 `0xbb`？请给出计算过程。（3 分）
7. 所给程序运行时 `printf` 函数的具体输出结果是什么？请详细分析输出结果的由来。（6 分）

得分	
----	--

二、针对题干和 `main.o` 与 `test.o` 的部分反汇编结果，回答下列问题。

1. 在 `main.o` 和 `test.o` 中，符号 `ar` 分别定义在哪个节中？在可执行文件 `test` 中，符号 `ar` 定义在虚拟地址空间中的哪个段内？占据多少字节？程序中多处定义了符号 `ar`，为何在程序链接时未报告错误？（6 分）

2. 列出所示 main.o 文件目标代码中需要重定位的所有符号引用的偏移量、目标符号名和重定位类型。（6 分）

引用偏移量	目标符号名	重定位类型

得分	
----	--

三、假设系统采用分页虚拟存储管理方式，页大小为 4KB，在运行可执行文件 test 对应的进程时，系统中没有其他进程在运行，回答下列问题。

1. 假设 test 函数中双重循环对应的首条机器指令“mov \$0x0,%esi”的虚拟地址为 0x80491a1，数组 ar 的起始虚拟地址为 0x804c040，则执行该双重循环的过程中，一共发生几次缺页异常？为什么？（5 分）

2. 若将 test.c 中结构 R 的两个字段 n1、n2 的类型改为 int，则重新编译、链接后生成的程序 test 在执行时显示“Segmentation fault”，请问程序执行过程中在数据访问时可能发生了什么问题？为什么会发生这种执行结果？（4 分）

得分	
----	--

四、假设系统中有一个数据区容量为 16KB 的数据 cache，采用 8 路组相联映射方式和 LRU 替换算法以及写回（Write Back）与写分配（Write Allocate）方式，主存块大小为 64B。test 中数组 ar 被分配在连续的物理主存单元中，且起始地址位于一个主存块的开始处。编译器将 test 函数中的变量 i、j 分配在通用寄存器中，数据 cache 初始为空，测得 test 程序执行时间为 0.012s。请回答下列问题。

1. 该数据 cache 共有多少行？主存地址中的组索引和块内地址字段分别占几位？（3 分）

2. test 函数中访问数组 ar 所得到的数据 cache 缺失率是多少？（4 分）

3. 如果将源文件 test.c 中第 7、8 两行进行交换（即按列优先访问 ar 数组元素），则在其它运行环境都不变的情况下测得其执行时间为 0.048s。采用按列优先访问后，在外循环变量 j 确定时内循环变量 i 遍历一遍共有多少次数据访存操作？其中发生多少次 cache 替换？test 函数中访问数组 ar 所得到的数据 cache 缺失率是多少？程序性能变差的原因是什么？（6 分）

得分	
----	--

五、针对第三题第 2 问中程序的运行结果，结合 IA-32+Linux 平台的异常处理机制，具体说明系统是如何检测、响应和处理该类异常事件的（300 字左右，10 分）。

得分	
----	--

六、选择题（每小题 2 分，共 20 分）

1. 考虑以下 C 语言代码：

```
unsigned char uc = -8;
```

```
short s = uc;
```

执行上述程序段后，s 的值是（ ）。

A. -8 B. 8 C. 248 D. 65528

2. 假设 R[*eax*]=0000 2023H，R[*ebx*]=2023 0012H，执行指令“mulw %*bx*”后，寄存器内容变化为（ ）。

A. R[*eax*]=0000 4276H，R[*dx*]=0000H

B. R[*eax*]=0002 4276H，R[*dx*]=0012H

C. R[*eax*]=0002 4276H，R[*dx*]=0000H

D. R[*eax*]=0000 4276H，R[*dx*]=0002H

3. 某 C 程序中定义了一个结构类型“struct T { char c; int n; short s; float f; }”，则 C 表达式“sizeof(struct T)”的值是（ ）。

A. 11

B. 12

C. 16

D. 20

4. 某 C 函数中包含语句“static int **p* = &*n*;”，其中 *n* 是一全局变量，则以下叙述中错误的是（ ）。

A. 上列语句使用 *n* 的地址对 *p* 进行初始化发生于函数首次执行时

B. 变量 *p* 的存储空间位于程序的可读写数据段中

- C. `.rel.data` 节中包含 `p` 的重定位信息
D. 编译该函数生成的可重定位目标文件的符号表中包含 `p` 的信息
5. 对于容量为 $16\text{M} \times 4$ 位的 DRAM 芯片，其地址引脚数和数据引脚数各是（ ）。
- A. 16 和 4 B. 16 和 1 C. 12 和 4 D. 64 和 1
6. 假设一个采用 8 路组相联映射方式的 TLB 具有 128 个页表项，用以查询 TLB 的虚页号表示为 20 位，则虚页号中有（ ）位是 TLB 标记。
- A. 10 B. 12 C. 14 D. 16
7. 以下操作中，由硬件完成而不是通过执行指令完成的是（ ）。
- A. 从 I/O 接口取数 B. 保护断点 C. 保护现场 D. 设置中断屏蔽字
8. 以下给出的选项中，不会引起从用户态陷入内核态的是（ ）。
- A. 发生 cache 缺失 B. 发生缺页 C. DMA 传送完成 D. 按 “Ctrl+C” 键
9. IA32+Linux 系统中，用户程序调用 `gets` 库函数从标准输入设备读入一个字符串的过程中，以下不同层软件的执行顺序是（ ）。
- I. 用户程序 II. `gets` 标准库函数 III. 系统调用服务例程(`sys_read`)
IV. 系统调用处理程序(`system_call`) V. 设备驱动程序 VI. `read()`系统调用封装函数
- A. I→II→III→IV→VI→V B. I→II→VI→V→IV→III
C. I→II→VI→IV→III→V D. I→II→IV→III→V→VI
10. 假设计算机系统中软盘以中断方式与 CPU 进行数据交换，主频为 100MHz ，传输单位为 32 位，软盘的数据传输率为 128KB/s 。若每次数据传输的开销（包括中断响应和中断处理）为 100 个时钟周期，则软盘工作时 CPU 用于软盘数据传输的时间占整个 CPU 时间的百分比是（ ）。
- A. 1% B. 3.2% C. 6.4% D. 10%